

Middleware & error handling

COMPLETE LEARNING GUIDE

What You'll Learn

Middleware is the mechanism that turns a web framework from a single request handler into a pipeline of composable, single-purpose functions — logging, parsing, authentication, and business logic all plug in as separate stages. This guide walks through what middleware actually is in the request-response cycle, how to write custom middleware and reason about execution order (which is a frequent source of subtle bugs), and — critically — how to centralize error handling so failures are caught, logged, and returned consistently rather than crashing the process or leaking internals. Special attention goes to the specific trap of async errors in Express, which don't behave the way synchronous errors do and silently disappear unless handled deliberately.

1. What Middleware Is and the Request-Response Cycle

In a framework like Express, a middleware function receives the request, response, and a `next()` function, and sits in a chain that a request passes through before reaching its final route handler. Each middleware can inspect or modify the request/response, end the cycle by sending a response, or call `next()` to pass control to the next function in the chain. This design lets cross-cutting concerns (logging every request, parsing JSON bodies, checking authentication) live as independent, reusable functions instead of being copy-pasted into every route handler.

2. Built-in and Third-Party Middleware

Most Express apps compose a handful of well-known middleware before any custom logic: `express.json()` parses JSON request bodies into `req.body`, `cors` handles cross-origin request headers, `helmet` sets a batch of security-related HTTP headers, and `morgan` logs each incoming request. Reaching for a well-maintained package for these common, well-understood concerns is almost always better than hand-rolling them, since the edge cases (character encoding, header spec compliance, security header nuances) have already been solved and battle-tested.

3. Writing Custom Middleware

Custom middleware follows the same (req, res, next) signature and is where app-specific cross-cutting logic lives: request logging with timing, attaching a request ID for tracing, or validating a header present on every request. Forgetting to call next() (when the middleware isn't itself ending the response) is the most common bug — the request simply hangs forever with no error, since nothing downstream is ever invoked.

JAVASCRIPT

```
function requestLogger(req, res, next) {
  const start = Date.now();
  res.on("finish", () => {
    const ms = Date.now() - start;
    console.log(`${req.method} ${req.originalUrl} ${res.statusCode} ${ms}ms`);
  });
  next(); // MUST be called or the request hangs
}

app.use(requestLogger);
```

4. Middleware Order Matters

Middleware executes in the exact order it's registered with app.use()/app.get() etc. — this isn't a stylistic preference, it changes behavior. Body-parsing middleware (express.json()) must run before any route handler that reads req.body, or req.body will be undefined. Authentication middleware must run before authorization middleware, which must run before the route handler doing the actual work. And error-handling middleware — a special case covered next — must be registered last, after every other app.use() and route definition, or it will never catch errors thrown by things registered after it.

5. Centralized Error-Handling Middleware

Express recognizes error-handling middleware by its distinct four-argument signature (err, req, res, next) and routes any error passed to next(err) — from anywhere in the app — directly to it, skipping all normal middleware in between. Centralizing error handling this way means every route can fail consistently: log the error once, decide the right status code, and return a client-safe error body, without duplicating that logic in every single route handler.

JAVASCRIPT

```
// Must be registered AFTER all routes and other middleware
function errorHandler(err, req, res, next) {
  console.error(err); // full details go to server logs

  const status = err.statusCode || 500;
  const message = status === 500 ? "Internal server error" : err.message;

  res.status(status).json({ error: { message } }); // client-safe body only
}

app.use(errorHandler);
```

6. Handling Async Errors Correctly

Express's built-in error handling only automatically catches errors thrown synchronously inside a route handler. An error thrown inside an async function's rejected Promise (e.g. an await'd database call that fails) does not automatically reach the error-handling middleware in older Express versions — it needs to be caught and explicitly forwarded with `next(err)`, or the request hangs with an unhandled rejection and no response is ever sent to the client. A small wrapper utility that catches rejected promises and forwards them to `next()` is the standard fix, avoiding a repetitive `try/catch` in every single async route.

JAVASCRIPT

```
function asyncHandler(fn) {
  return (req, res, next) => {
    Promise.resolve(fn(req, res, next)).catch(next);
  };
}

app.get("/api/students/:id", asyncHandler(async (req, res) => {
  const student = await db.student.findUnique({ where: { id: req.params.id } });
  if (!student) {
    const err = new Error("Student not found");
    err.statusCode = 404;
    throw err; // caught by asyncHandler, forwarded to errorHandler
  }
  res.json(student);
})));
```

7. Validation as Middleware

Input validation is a natural fit for middleware, since it's a cross-cutting concern that should run before a route handler ever executes its business logic. Libraries like `Zod` or `Joi` let you define a schema once and validate `req.body/req.query/req.params` against it in a single reusable middleware, rejecting malformed requests with a clear 400 response before any database call or business logic runs — rather than scattering ad hoc if-checks throughout the handler, or worse, letting invalid data reach the database layer.

Common Pitfalls

- Forgetting to call `next()` in a middleware that isn't ending the response, causing the request to hang indefinitely with no error surfaced.
- Registering error-handling middleware before other routes/middleware instead of last, so it never catches errors from anything registered after it.
- Not catching rejected promises in async route handlers, leaving requests hanging with unhandled rejections and no response sent.
- Registering body-parsing middleware after the routes that need `req.body`, so `req.body` is undefined in those handlers.
- Building one large middleware that does logging, auth, and validation all at once instead of small, single-purpose, composable ones.
- Returning raw `error.stack` or internal exception details to the client instead of a generic, client-safe error message.
- Not distinguishing operational errors (expected, like 'not found' or bad input) from programmer errors (bugs) when deciding what to log versus what to alert on.

Key Takeaways

- Middleware forms an ordered pipeline; execution order changes behavior, not just readability.
- Body-parsing must run before handlers that read `req.body`; auth must run before authorization; error handlers must be registered last.
- Express only auto-catches synchronous errors — async/Promise-based errors must be explicitly forwarded to `next(err)`.
- Centralized error-handling middleware keeps logging and client-facing error shape consistent across every route.
- Never return raw stack traces or internal error details to API clients — log them server-side and return a generic, safe message.
- Validation belongs in middleware, running before business logic, not scattered as inline checks inside handlers.

Learn More

Express — Using Middleware

<https://expressjs.com/en/guide/using-middleware.html>

Express — Error Handling

<https://expressjs.com/en/guide/error-handling.html>

Node.js Docs — Error Handling Best Practices

<https://nodejs.org/en/learn/asynchronous-work/discover-promises-in-nodejs>